

UserSide/backends/encryptionService.js

```
1: // AES helper for sensitive fields.
2: // Works with both Node format and Laravel format.
3:
4: const crypto = require('crypto');
5:
6: // Matches Laravel's default cipher.
7: const ALGORITHM = 'aes-256-cbc';
8: const ENCRYPTION_KEY = process.env.APP_KEY || 'base64:ciPqFYtQJ2bGZ0NurfY7mvwODu0Z6zyUT1Ih1D+pb+w=';
9:
10: // Get the key from APP_KEY.
11: // If APP_KEY is base64:..., decode it first.
12: function getEncryptionKey() {
13:   let key = ENCRYPTION_KEY;
14:
15:   // Decode Laravel-style base64 key.
16:   if (key.startsWith('base64:')) {
17:     key = key.substring(7);
18:     return Buffer.from(key, 'base64');
19:   }
20:
21:   // Fallback: normalize to 32 bytes for AES-256.
22:   return Buffer.from(key.padEnd(32, '0').substring(0, 32));
23: }
24:
25: // Encrypt plain text using AES-256-CBC.
26: // Returns base64(iv + ciphertext).
27: function encrypt(text) {
28:   if (!text) return text;
29:
30:   try {
31:     // Random 16-byte IV.
32:     const iv = crypto.randomBytes(16);
33:
34:     const key = getEncryptionKey();
35:
36:     const cipher = crypto.createCipheriv(ALGORITHM, key, iv);
37:
38:     let encrypted = cipher.update(text, 'utf8', 'base64');
39:     encrypted += cipher.final('base64');
40:
41:     // Output format: base64(iv + ciphertext).
42:     const combined = Buffer.concat([iv, Buffer.from(encrypted, 'base64')]);
43:
44:     return combined.toString('base64');
45:   } catch (error) {
46:     console.error('Encryption error:', error.message);
47:     throw new Error('Failed to encrypt data');
48:   }
49: }
50:
51: // Try to decrypt one layer.
52: // If not decryptable, just return the original input.
53: function decryptOnce(encryptedData) {
54:   if (!encryptedData) return encryptedData;
55:
56:   // Quick guard for non-encrypted values.
57:   if (typeof encryptedData !== 'string' || encryptedData.length < 24) {
58:     return encryptedData;
59:   }
60:
61:   try {
62:     const key = getEncryptionKey();
63:
64:     // Try Laravel payload format first.
```

UserSide/backends/encryptionService.js (continued)

```
65:     try {
66:         const decoded = Buffer.from(encryptedData, 'base64').toString('utf8');
67:         const payload = JSON.parse(decoded);
68:         if (payload && payload.iv && payload.value) {
69:             const iv = Buffer.from(payload.iv, 'base64');
70:             const encrypted = Buffer.from(payload.value, 'base64');
71:             const decipher = crypto.createDecipheriv(ALGORITHM, key, iv);
72:             let decrypted = decipher.update(encrypted, undefined, 'utf8');
73:             decrypted += decipher.final('utf8');
74:             // Strip PHP serialized wrapper if present.
75:             const phpMatch = decrypted.match(/^s:\d+:"(.*)"$/s);
76:             if (phpMatch) return phpMatch[1];
77:             return decrypted;
78:         }
79:     } catch (laravelError) {
80:         // Not Laravel format.
81:     }
82:
83:     // Try Node format: base64(iv + ciphertext).
84:     let combined;
85:     try {
86:         combined = Buffer.from(encryptedData, 'base64');
87:     } catch (decodeError) {
88:         return encryptedData;
89:     }
90:
91:     // Need at least IV (16 bytes) + 1 byte payload.
92:     if (combined.length < 17) {
93:         return encryptedData;
94:     }
95:
96:     // Extract IV and ciphertext.
97:     const iv = combined.slice(0, 16);
98:     const encrypted = combined.slice(16);
99:
100:    const decipher = crypto.createDecipheriv(ALGORITHM, key, iv);
101:
102:    let decrypted = decipher.update(encrypted, undefined, 'utf8');
103:    decrypted += decipher.final('utf8');
104:
105:    return decrypted;
106: } catch (error) {
107:     // Keep original value if decryption fails.
108:     return encryptedData;
109: }
110: }
111:
112: // Fully decrypt text.
113: // Some old values were encrypted multiple times, so we unwrap up to 20 layers.
114: function decrypt(encryptedData) {
115:     if (!encryptedData) return encryptedData;
116:     if (typeof encryptedData !== 'string' || encryptedData.length < 24) return encryptedData;
117:
118:     let current = encryptedData;
119:     for (let i = 0; i < 20; i++) {
120:         const decrypted = decryptOnce(current);
121:         if (decrypted === current) break;
122:         current = decrypted;
123:     }
124:     return current;
125: }
126:
127: // Encrypt file bytes.
128: // Output format is IV + encrypted bytes.
```

UserSide/backends/encryptionService.js (continued)

```
129: function encryptFile(fileBuffer) {
130:   if (!fileBuffer) return fileBuffer;
131:
132:   try {
133:     const iv = crypto.randomBytes(16);
134:     const key = getEncryptionKey();
135:     const cipher = crypto.createCipheriv(ALGORITHM, key, iv);
136:
137:     const encrypted = Buffer.concat([cipher.update(fileBuffer), cipher.final()]);
138:
139:     // Prefix IV for later decryption.
140:     return Buffer.concat([iv, encrypted]);
141:   } catch (error) {
142:     console.error(' File encryption error:', error.message);
143:     throw new Error('Failed to encrypt file');
144:   }
145: }
146:
147: // Decrypt file bytes.
148: // If decrypt fails, return original bytes so we don't break file handling.
149: function decryptFile(encryptedBuffer) {
150:   if (!encryptedBuffer) return encryptedBuffer;
151:
152:   // Need IV (16 bytes) + 1 byte payload.
153:   if (encryptedBuffer.length < 17) {
154:     console.log(' File too small to decrypt, returning as-is');
155:     return encryptedBuffer;
156:   }
157:
158:   try {
159:     const key = getEncryptionKey();
160:
161:     // Extract IV and ciphertext.
162:     const iv = encryptedBuffer.slice(0, 16);
163:     const encrypted = encryptedBuffer.slice(16);
164:
165:     const decipher = crypto.createDecipheriv(ALGORITHM, key, iv);
166:
167:     return Buffer.concat([decipher.update(encrypted), decipher.final()]);
168:   } catch (error) {
169:     // If decrypt fails, keep original buffer.
170:     console.log(' File decryption failed, returning original (likely not encrypted)');
171:     return encryptedBuffer;
172:   }
173: }
174:
175: // Encrypt only selected fields in an object.
176: function encryptFields(obj, fields) {
177:   const encrypted = { ...obj };
178:
179:   fields.forEach(field => {
180:     if (encrypted[field]) {
181:       encrypted[field] = encrypt(String(encrypted[field]));
182:     }
183:   });
184:
185:   return encrypted;
186: }
187:
188: // Decrypt only selected fields in an object.
189: function decryptFields(obj, fields) {
190:   const decrypted = { ...obj };
191:
192:   fields.forEach(field => {
```

UserSide/backends/encryptionService.js (continued)

```
193:     if (decrypted[field]) {
194:         decrypted[field] = decrypt(decrypted[field]);
195:     }
196: });
197:
198: return decrypted;
199: }
200:
201: // Simple role check for who can view decrypted values.
202: function canDecrypt(userRole) {
203:     const authorizedRoles = ['police', 'admin'];
204:     return authorizedRoles.includes(userRole);
205: }
206:
207: module.exports = {
208:     encrypt,
209:     decrypt,
210:     encryptFile,
211:     decryptFile,
212:     encryptFields,
213:     decryptFields,
214:     canDecrypt,
215:     ALGORITHM
216: };
```

AdminSide/admin/app/Services/EncryptionService.php

```
1: <?php
2:
3: namespace App\Services;
4:
5: use Illuminate\Support\Facades\Crypt;
6: use Illuminate\Contracts\Encryption\DecryptException;
7: use Illuminate\Support\Facades\Log;
8:
9: // AES helper for AdminSide.
10: // Supports both Laravel and Node-style encrypted values.
11: class EncryptionService
12: {
13:     // Read APP_KEY. If it's base64:..., decode it.
14:     private static function getEncryptionKey()
15:     {
16:         $key = config('app.key');
17:
18:         // If key starts with "base64:", decode it
19:         if (str_starts_with($key, 'base64:')) {
20:             return base64_decode(substr($key, 7));
21:         }
22:
23:         return $key;
24:     }
25:
26:     // Encrypt text with Laravel Crypt.
27:     public static function encrypt($text)
28:     {
29:         if (empty($text)) {
30:             return $text;
31:         }
32:
33:         try {
34:             // Laravel encryption (AES-256-CBC by default)
35:             return Crypt::encryptString($text);
36:         } catch (\Exception $e) {
37:             Log::error('Encryption error: ' . $e->getMessage());
38:             throw new \Exception('Failed to encrypt data');
39:         }
40:     }
41:
42:     // Decrypt Node format: base64(iv + ciphertext).
43:     private static function decryptNodeJsFormat($encryptedData)
44:     {
45:         try {
46:             // Decode input
47:             $combined = base64_decode($encryptedData, true);
48:             if ($combined === false) {
49:                 return null;
50:             }
51:
52:             // Need at least IV (16 bytes) + 1 byte payload
53:             if (strlen($combined) < 17) {
54:                 return null;
55:             }
56:
57:             // Split into IV and ciphertext
58:             $iv = substr($combined, 0, 16);
59:             $encrypted = substr($combined, 16);
60:
61:             // Try main key first
62:             $key = self::getEncryptionKey();
63:
64:             $decrypted = openssl_decrypt(
```

AdminSide/admin/app/Services/EncryptionService.php (continued)

```
65:         $encrypted,
66:         'aes-256-cbc',
67:         $key,
68:         OPENSSL_RAW_DATA,
69:         $iv
70:     );
71:
72:     if ($decrypted !== false) {
73:         return $decrypted;
74:     }
75:
76:     // Fallback key for cross-service APP_KEY mismatch
77:     $fallbackKey = base64_decode('ciPqFYTQJ2bGZ0NURfY7mvwODuOZ6zyUT1Ih1D+pb+w=');
78:     if ($fallbackKey !== $key) {
79:         $decrypted = openssl_decrypt(
80:             $encrypted,
81:             'aes-256-cbc',
82:             $fallbackKey,
83:             OPENSSL_RAW_DATA,
84:             $iv
85:         );
86:
87:         if ($decrypted !== false) {
88:             return $decrypted;
89:         }
90:     }
91:
92:     // Clear OpenSSL error queue
93:     while (openssl_error_string()) {}
94:     return null;
95: } catch (\Exception $e) {
96:     return null;
97: }
98: }
99:
100: // Decrypt text. Handles Laravel + Node formats and multi-layer values.
101: public static function decrypt($encryptedText)
102: {
103:     if (empty($encryptedText) || !is_string($encryptedText)) {
104:         return $encryptedText;
105:     }
106:
107:     try {
108:         // Unwrap up to 20 layers (old data may be encrypted multiple times)
109:         $current = $encryptedText;
110:         for ($i = 0; $i < 20; $i++) {
111:             $decrypted = self::decryptOnce($current);
112:             if ($decrypted === $current) break;
113:             $current = $decrypted;
114:         }
115:         return $current;
116:     } catch (\Exception $e) {
117:         Log::error('Unexpected error during decryption', [
118:             'error' => $e->getMessage(),
119:             'data_preview' => substr($encryptedText, 0, 50)
120:         ]);
121:         return $encryptedText;
122:     }
123: }
124:
125: // Try one decrypt pass: Laravel first, then Node format.
126: private static function decryptOnce($encryptedText)
127: {
128:     if (empty($encryptedText) || !is_string($encryptedText)) {
```

AdminSide/admin/app/Services/EncryptionService.php (continued)

```
129:         return $encryptedText;
130:     }
131:
132:     try {
133:         // Try Laravel payload first
134:         try {
135:             return Crypt::decryptString($encryptedText);
136:         } catch (\Exception $e) {
137:             // Not Laravel format
138:         }
139:
140:         $decrypted = self::decryptNodeJsFormat($encryptedText);
141:         if ($decrypted !== null) {
142:             return $decrypted;
143:         }
144:
145:         // If both fail, likely plaintext
146:         return $encryptedText;
147:     } catch (\Exception $e) {
148:         return $encryptedText;
149:     }
150: }
151:
152: // Encrypt selected fields in an array.
153: public static function encryptFields(array $data, array $fields)
154: {
155:     foreach ($fields as $field) {
156:         if (isset($data[$field]) && !empty($data[$field])) {
157:             $data[$field] = self::encrypt($data[$field]);
158:         }
159:     }
160:
161:     return $data;
162: }
163:
164: // Decrypt selected fields in an array.
165: public static function decryptFields(array $data, array $fields)
166: {
167:     foreach ($fields as $field) {
168:         if (isset($data[$field]) && !empty($data[$field])) {
169:             $data[$field] = self::decrypt($data[$field]);
170:         }
171:     }
172:
173:     return $data;
174: }
175:
176: // Decrypt selected fields on an object/model.
177: public static function decryptModelFields($model, array $fields)
178: {
179:     foreach ($fields as $field) {
180:         if (isset($model->$field) && !empty($model->$field)) {
181:             $model->$field = self::decrypt($model->$field);
182:         }
183:     }
184:
185:     return $model;
186: }
187:
188: // Role check for who can view decrypted values.
189: public static function canDecrypt($userRole)
190: {
191:     $authorizedRoles = ['police', 'admin', 'super_admin'];
192:     return in_array($userRole, $authorizedRoles);
```

AdminSide/admin/app/Services/EncryptionService.php (continued)

```
193:     }
194:
195:     // Best-effort check if a value looks encrypted.
196:     public static function isEncrypted($text)
197:     {
198:         if (empty($text) || !is_string($text)) {
199:             return false;
200:         }
201:
202:         // Laravel format: base64(JSON with iv/value/mac)
203:         try {
204:             $decoded = base64_decode($text, true);
205:             if ($decoded !== false) {
206:                 $json = json_decode($decoded, true);
207:                 if (is_array($json) && isset($json['iv']) && isset($json['value']) && isset($json['mac'])) {
208:                     return true;
209:                 }
210:             }
211:         } catch (\Exception $e) {
212:             // Not Laravel format
213:         }
214:
215:         // Node format: base64(iv + ciphertext)
216:         // 16-byte IV + at least 16-byte block => ~44 base64 chars minimum
217:         if (strlen($text) >= 44) {
218:             $decoded = base64_decode($text, true);
219:             if ($decoded !== false && strlen($decoded) >= 32) {
220:                 // Encrypted binary usually has non-printable bytes
221:                 $nonPrintable = preg_match('/^[^\x20-\x7E]/', $decoded);
222:                 if ($nonPrintable) {
223:                     return true;
224:                 }
225:             }
226:         }
227:
228:         return false;
229:     }
230: }
```